EEE485 Term Project: First Report

Deniz Çınar Bayburt - 22201822
Osman Gök- 21801997

March 27, 2026

Introduction

Problem Description

The goal of this project is to predict the outcomes of English Premier League matches using only pre-match information. Rather than using statistics from the match being predicted, our constructs each input vector from the historical performance of the two teams. More specifically, for a given fixture, the home team is represented by its previous home match statistics, and the away team is represented by its previous away-match statistics.

A major challenge in football prediction is data leakage. If post-match statistics from the target match are used as inputs, the prediction problem becomes unrealistic. To avoid this issue, our entire pipeline is designed so that every feature is computed strictly from matches that occurred before the target match. This makes the model suitable for real pre-match forecasting rather than retrospective classification.

The project is structured around implementing a baseline multiclass prediction system using Logistic Regression. The baseline predicts three classes: Home Win (H), Draw (D), and Away Win (A). This baseline will later be compared against more advanced models proposed in the project, such as XGBoost and Multi-Layer Perceptrons (MLP).

Review of Proposed Methods

As described in the original project proposal, the final project is planned to include multiple machine learning methods. At this stage, the first baseline model has been implemented and analyzed.

- **Logistic Regression (Implemented)**: Logistic Regression serves as the baseline model for the project. It is a linear classifier, which makes it interpretable and suitable for understanding the effect of feature engineering. In this project, three versions were implemented: standard logistic regression, ridge-regularized logistic regression, and lasso-regularized logistic regression.

- **XGBoost (Future Work):** XGBoost is expected to improve performance because it can capture nonlinear relationships and feature interactions that logistic regression cannot model effectively.

- **Multi-Layer Perceptron (Future Work):** An MLP will be used as another advanced method for modeling more complex decision boundaries in football outcome prediction.

Expected Challenges

Several major challenges were identified in the project, and many of them were already encountered during implementation.

- **Data Leakage:** This was the primary challenge. It was addressed by constructing each match vector only from previous matches. The current match's shots, goals, fouls, or cards are never used as input features.
- **Feature Engineering:** Raw match results alone are not sufficient for robust prediction. The main challenge was to design features that represent team strength, recent form, and discipline separately for home and away contexts.
- **Multiclass Prediction:** Predicting H/D/A is more difficult than binary classification because the draw class is harder to separate from the other two classes.
- **Custom Model Implementation:** The logistic regression models were implemented directly rather than relying entirely on prebuilt end-to-end model pipelines. This required implementing the sigmoid function, the loss function, iterative optimization, regularization handling, and evaluation logic manually.
- **Class Imbalance and Draw Prediction:** Draws are less frequent and more difficult to model. This makes the multiclass setting significantly harder, especially for linear models**.**

Dataset and Preprocessing

Dataset Description

The dataset is a compilation of 10 separate CSV files from GitHub spanning from the 2015/16 to the 2024/25 seasons [1]. After concatenation, the full dataset contains 3,800 matches.

Each row represents a single match and contains post-match statistics (shots, shots on target, fouls, away team, red card, result, etc.). However, our aim is to predict the match before the match. Therefore, any information about the match results cannot be used. As a result, the data is arranged into two main categories: away team statistics and home team statistics for the last 5 or 15 (depending on the parameter type) matches of the teams playing the match. However, in some seasons, some of the teams did not play or played for the first time. As a result, this caused a data leakage in our model. To avoid this problem, we extract matches in which at least one team had no information about their last 15 games. Note that this ensures the model uses data from the team that played at least 15 games before the current match.

Moreover, after arranging the statistics for the last 5 or 15 matches of the teams and preventing data leakage, the X matrix can be found on the GitHub pages of Deniz Çınar Bayburt and Osman Gök. The total number of matches becomes 3728. The parameters are listed below:

- **x_0:** The model's constant (bias/intercept) term
- **home_gf_last15_avg**: Home team's average Goals For (goals scored) per game over their last 15 matches
- **home_gf_last5_avg**: Home team's average Goals For per game over their last 5 matches
- **home_shot_on_target_last15_avg**: Home team's average shots on target per game over their last 15 matches
- **home_shot_on_target_last5_avg:** Home team's average shots on target per game over their last 5 matches
- **home_shot_last15_avg:** Home team's average total shots per game over their last 15 matches

- **home_shot_last5_avg:** Home team's average total shots per game over their last 5 matches
- **home_corner_last15_avg:** Home team's average corners won per game over their last 15 matches
- **home_corner_last5_avg:** Home team's average corners won per game over their last 5 matches
- **home_faul_againts_last15_avg:** Home team's average fouls suffered (fouls committed by the opponent) per game over their last 15 matches
- **home_faul_againts_last5_avg:** Home team's average fouls suffered per game over their last 5 matches
- **home_ga_last5_avg:** Home team's average Goals Against (goals conceded) per game over their last 5 matches
- **home_zga_last5_avg:** Home team's average Zero Goals Against (Clean Sheet) ratio over their last 5 matches
- **home_ga_last15_avg:** Home team's average Goals Against per game over their last 15 matches
- **home_shot_on_target_against_last5_avg:** Home team's average shots on target faced per game over their last 5 matches
- **home_shot_on_target_against_last15_avg:** Home team's average shots on target faced per game over their last 15 matches
- **home_shot_against_last5_avg:** Home team's average total shots faced per game over their last 5 matches
- **home_shot_against_last15_avg:** Home team's average total shots faced per game over their last 15 matches
- **home_faul_last5_avg:** Home team's average fouls committed per game over their last 5 matches
- **home_faul_last15_avg:** Home team's average fouls committed per game over their last 15 matches
- **home_card_points_last15_avg:** Home team's average booking/card points (e.g., Yellow=10, Red=25) per game over their last 15 matches
- **home_card_points_last5_avg:** Home team's average card points per game over their last 5 matches
- **home_cord_points_per_faul_last15_avg:** Home team's average card points per foul committed over their last 15 matches (an index of aggression/recklessness)
- **home_cord_points_per_faul_last5_avg:** Home team's average card points per foul committed over their last 5 matches
- **away_gf_last15_avg:** Away team's average Goals For (goals scored) per game over their last 15 matches
- **away_gf_last5_avg:** Away team's average Goals For per game over their last 5 matches
- **away_shot_on_target_last15_avg:** Away team's average shots on target per game over their last 15 matches
- **away_shot_on_target_last5_avg:** Away team's average shots on target per game over their last 5 matches
- **away_shot_last15_avg:** Away team's average total shots per game over their last 15 matches
- **away_shot_last5_avg:** Away team's average total shots per game over their last 5 matches
- **away_corner_last15_avg:** Away team's average corners won per game over their last 15 matches
- **away_corner_last5_avg:** Away team's average corners won per game over their last 5 matches
- **away_faul_againts_last15_avg:** Away team's average fouls suffered per game over their last 15 matches
- **away_faul_againts_last5_avg:** Away team's average fouls suffered per game over their last 5 matches
- **away_ga_last5_avg:** Away team's average Goals Against (goals conceded) per game over their last 5 matches
- **away_zga_last5_avg:** Away team's average Zero Goals Against (Clean Sheet) ratio over their last 5 matches
- **away_ga_last15_avg:** Away team's average Goals Against per game over their last 15 matches
- **away_shot_on_target_against_last5_avg:** Away team's average shots on target faced per game over their last 5 matches
- **away_shot_on_target_against_last15_avg:** Away team's average shots on target faced per game over their last 15 matches

- **away_shot_against_last5_avg:** Away team's average total shots faced per game over their last 5 matches
- **away_shot_against_last15_avg:** Away team's average total shots faced per game over their last 15 matches
- **away_faul_last5_avg:** Away team's average fouls committed per game over their last 5 matches
- **away_faul_last15_avg:** Away team's average fouls committed per game over their last 15 matches
- **away_card_points_last15_avg:** Away team's average card points per game over their last 15 matches
- **away_card_points_last5_avg:** Away team's average card points per game over their last 5 matches
- **away_cord_points_per_faul_last15_avg:** Away team's average card points per foul committed over their last 15 matches
- **away_cord_points_per_faul_last5_avg:** Away team's average card points per foul committed over their last 5 matches

Data Splitting and Preprocessing

To prevent lucky or unlucky results, we implement cross-validation. Therefore, we split the data into 8 pieces. While training the model, we experiment 8 times for each regression model. For example, for the first time, we trained our model on the last 7 pieces and tested it on the first pieces. Then we trained our model without the second piece and tested it with it, and so on.

After converting the result vector ‘ y ’ “H/D/A“ format into a “0/1/2“ format, the final processed data shapes were:

- **X:** $[X_1 \quad X_2 \quad X_3 \quad X_4 \quad X_5 \quad X_6 \quad X_7 \quad X_8]$ where X_i has 466 rows and 47 features. Note that while training the model, we used $X_i^{train} = \begin{pmatrix} X \\ 7 \end{pmatrix}$ and X_i^{test} is the remaining element of X. Therefore, X_i^{train} has 3262 rows, 47 features, and X_i^{test} has 466 rows and 47 features.
- **Y:** $[Y_1 \quad Y_2 \quad Y_3 \quad Y_4 \quad Y_5 \quad Y_6 \quad Y_7 \quad Y_8]$ where Y_i has 466 rows and 1 feature (0, 1, or 2). Note that Y_i is corresponds the results of X_i^{test}.

Methodology

In theoretical problems, the dimensions of the input vector X are always assumed to be independent of each other. However, in real-life models, the parameters may not be independent of each other. Therefore, we created a correlation matrix. The equation can be seen in Equation 1:

$$\rho_{X,Y} = \frac{\mathrm{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X-\mu_X)(Y-\mu_Y)]}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i-\bar{x})^2 \sum_{i=1}^{n}(y_i-\bar{y})^2}} \quad (1)$$

As a result, we created the matrix according to Equation 2:

$$R = \begin{pmatrix} 1 & \rho_{12} & \cdots & \rho_{1p} \\ \rho_{21} & 1 & \cdots & \rho_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{p1} & \rho_{p2} & \cdots & 1 \end{pmatrix} \quad (2)$$

Therefore, we obtained the covariance matrix by using our data, which can be seen in Figure 1:

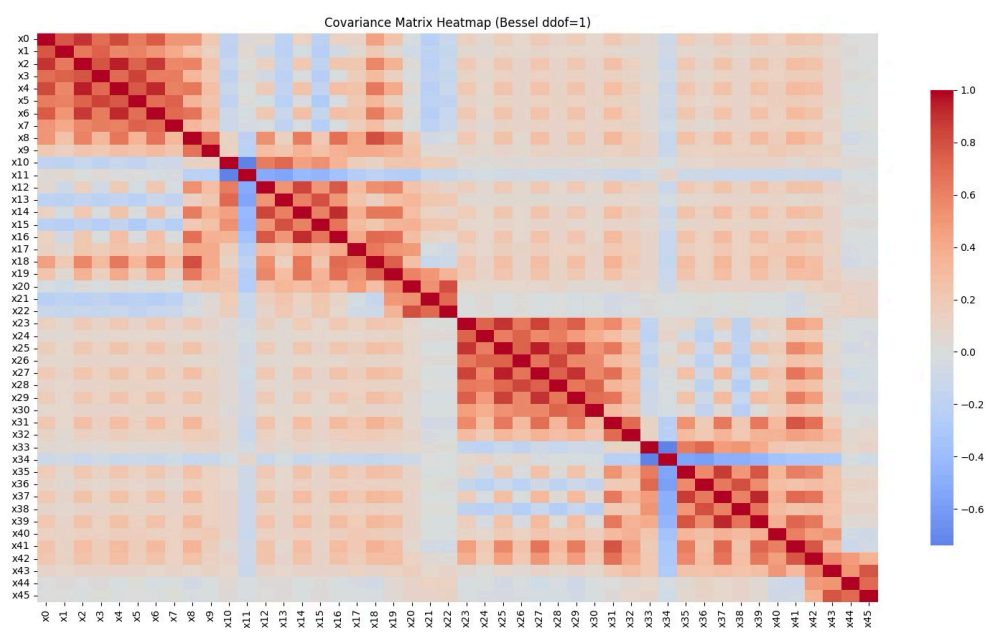


Fig 1: Covariance Matrix Heatmap

Note that while creating the covariance matrix, we did not include the bias term. Therefore, the number of real parameters is 46.

Logistic Regression

As required, we implemented Logistic Regression using only the numpy library. The algorithm consists of three core components. However, the results can be 0, 1, or 2. Therefore, we used logistic regressions for 3 times for each prediction. Therefore, we arranged the data as follows:

- For the first regression, the 0's are formed to be '1' while others are formed to be '0'.
- For the first regression, the 1's are formed to be '1' while others are formed to be '0'.
- For the first regression, the 2's are formed to be '1' while others are formed to be '0'.

As a result, we chose the result by looking at the highest probability produced by these logistic regressions for each match.

Moreover, to see the effects of regulators, we compared L1 and L2 regulators on the logistic regression.

1. The Sigmoid Function

This function takes the weighted sum of our features (a score z) and converts it into a probability p between 0 and 1.

$$p = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (3)$$

where $z_i = x_i \cdot \beta$ in general.

2. The Cost Function (Log Loss)

To measure the model's error, we use the Log Loss function. This function heavily penalizes the model for being "confidently wrong" (e.g., predicting 99% for a loss). The goal of training is to minimize this cost.

$$J(\beta, b) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(p^{(i)}) + (1 - y^{(i)})\log(1 - p^{(i)})] \quad (4)$$

where m is the number of matches, y is the actual outcome (0 or 1), and p is our model's predicted probability. However, if a regulator is used, the log loss will become:

- L2 (Ridge): $J(\boldsymbol{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln(p_i) + (1 - y_i)\ln(1 - p_i)\right] + \lambda\sum_{j=1}^{p}\beta_j^2 \quad (4.1)$
- L1(Lasso): $J(\boldsymbol{\beta}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \ln(p_i) + (1 - y_i)\ln(1 - p_i)\right] + \lambda\sum_{j=1}^{p}|\beta_j| \quad (4.2)$

3. Gradient Descent

This is the "learning" algorithm. It calculates the slope (gradient) of the cost function with respect to each feature weight, and then "steps" the weights in the opposite direction (downhill) to minimize the cost. We repeat this process for a set number of iterations.

$$\beta_j := \beta_j - \alpha\frac{\partial J}{\partial \beta_j}, \quad where \quad \frac{\partial J}{\partial \beta_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)x_{ij} \quad (5.1)$$

$$b := b - \alpha \frac{\partial J}{\partial b} \quad where \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} (p_i - y_i) \quad (5.2)$$

where α is the learning rate. However, when using the Lasso regulator, gradient descent cannot be applied because the log loss function contains an absolute value. Specifically, the L1 penalty term |βj| is not differentiable at βj =0, which prevents the calculation of a standard gradient at that point. Therefore, we implemented the Coordinate Descent algorithm combined with the Soft-Thresholding operator. This approach allows us to optimize each coefficient individually by bypassing the non-differentiability issue, effectively forcing less significant feature weights to exactly zero.

$$\beta_j = \frac{1}{\text{payda}} S\left(\rho_j, n\lambda\right) \quad (5.3)$$

$$S(\rho, \gamma) = \begin{cases} \rho - \gamma & \text{if } \rho > \gamma \\ \rho + \gamma & \text{if } \rho < -\gamma \\ 0 & \text{if } |\rho| \leq \gamma \end{cases} \quad (5.4)$$

$$\rho_j = \sum_{i=1}^{n} w_i x_{ij} \left(z_i - \sum_{k \neq j} x_{ik} \beta_k \right) \quad (5.5)$$

$$w_i = p_i(1 - p_i) \quad (5.6)$$

Simulation Setup and Results

Simulation Environment

All simulations were run in a Python 3 environment on VSCode. The only libraries used for the core logic were pandas (for data manipulation) and numpy (for the algorithm). Seaborn and Matplotlib were used only for visualization.

1. Training Process

The model was trained on 3278 match samples from the 2015/16 to the 2024/25 season, 8 times for each regression, and with different regulators. We used a learning rate (α) of 0.1 for ridge and lasso and 1000 iterations.

- **Total Feature Engineering Time (STEP 1):** 4.5 seconds
- **Total Algorithm Training Time (STEP 2):** 11.74 seconds

Figure 2 shows the model's learning process. The cost (Log Loss) successfully converged from an initial value of 0.6931 (random guessing) to a stable minimum of 0.6393, demonstrating that the gradient descent and coordinate descent algorithms worked correctly.

```
=== LOSS VS ITERATION SUMMARY (Fold 0) ===
Lasso: iterations=215, initial_loss=0.574755, final_loss=0.565878
Ridge: iterations=1000, initial_loss=0.691397, final_loss=0.566016
Standard: iterations=1000, initial_loss=0.691397, final_loss=0.565866
```

Fig 2.1: Loss vs Iteration (numerical)

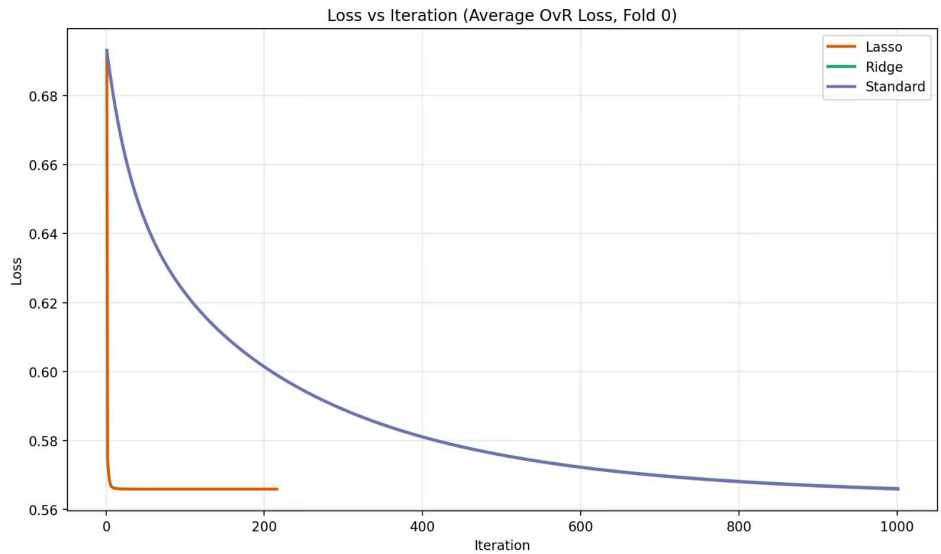


Fig 2.2: Loss vs Iteration

Performance Evaluation

The trained model was then evaluated on the unseen 466 matches for each process. The preliminary results are highly successful for a baseline model.

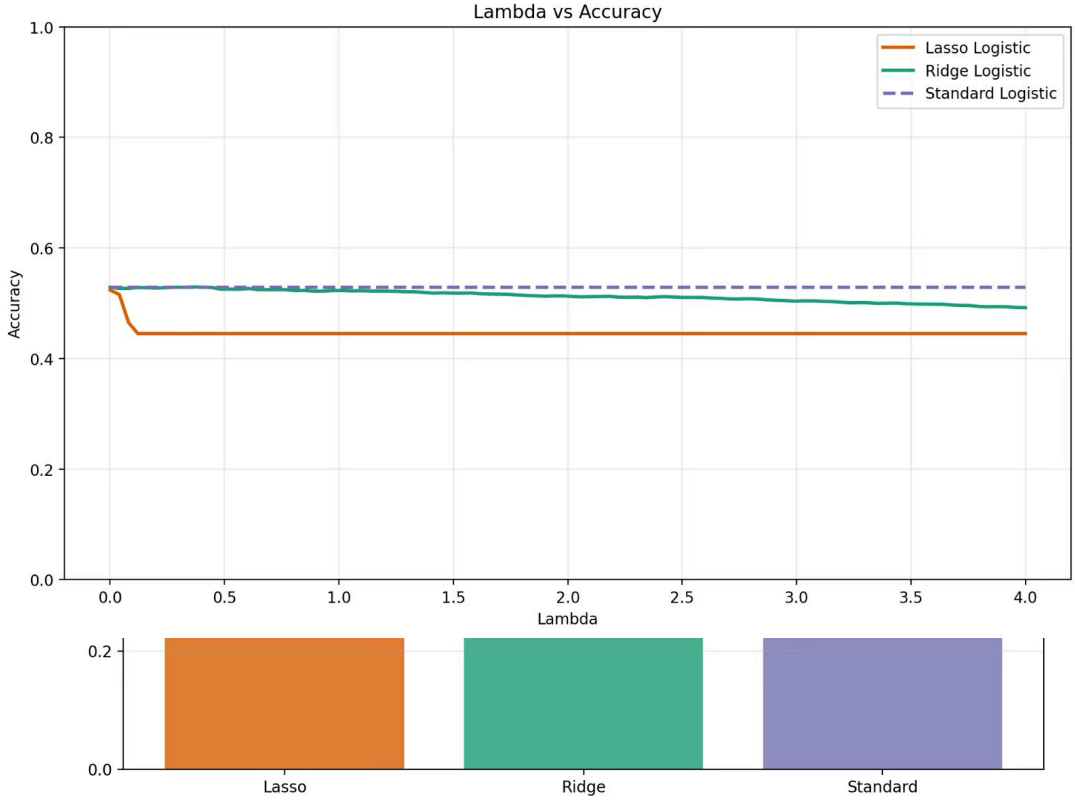
```
Fold 0: Lasso Logit: 0.4850, Ridge Logit: 0.4764, Standard Logit: 0.4764
Fold 1: Lasso Logit: 0.5773, Ridge Logit: 0.5815, Standard Logit: 0.5837
Fold 2: Lasso Logit: 0.5322, Ridge Logit: 0.5258, Standard Logit: 0.5258
Fold 3: Lasso Logit: 0.5172, Ridge Logit: 0.5129, Standard Logit: 0.5129
Fold 4: Lasso Logit: 0.5086, Ridge Logit: 0.5129, Standard Logit: 0.5129
Fold 5: Lasso Logit: 0.5300, Ridge Logit: 0.5300, Standard Logit: 0.5300
Fold 6: Lasso Logit: 0.5494, Ridge Logit: 0.5451, Standard Logit: 0.5451
Fold 7: Lasso Logit: 0.5279, Ridge Logit: 0.5451, Standard Logit: 0.5451

Average Accuracy -> Lasso: 0.5284, Ridge: 0.5287, Standard: 0.5290

=== 95% CONFIDENCE INTERVALS ===
Lasso: mean=0.5284, CI=[0.5055, 0.5513], half-width=0.0229, n=8
Ridge: mean=0.5287, CI=[0.5031, 0.5543], half-width=0.0256, n=8
Standard: mean=0.5290, CI=[0.5029, 0.5550], half-width=0.0261, n=8
```

Fig 3: The comparison of logistic regressions and their %95 confidence intervals

Fig 4: The 95% confidence intervals of regressions



Moreover, we compare the regulators with different λ values. The result can be seen in Figure 5:

Fig 5: The comparison of the regulators

Discussion of Results

The results are very strong. We obtained [0.5055 - 0.5513] 95% confidence interval for lasso logistic regression, [0.5031 - 0.5543] 95% confidence interval for ridge logistic regression, and [0.5029 - 0.5550] 95% confidence interval for standard logistic regression. This means that 33% prediction probability is upgraded to above of 50% for all regulators. The most important result is the relationship between the Training and Test Log Loss. While standard and ridge regression cannot minimize the loss function after 1000 iterations, meaning they do not overfit the test/train split, the lasso logistic regression algorithm prevents overfitting. Another feature of the regulators, λ, causes a decrease in the success rate. The reason behind this result is that the best success rate of the logistic regression is the standard logistic regression, which λ is equal to 0.

The whole work can be followed on https://github.com/OsmixG/Premier-League-Predictor or https://github.com/DenizBayburt

Group Member Contributions

This project was a collaborative effort. The main tasks were divided as follows.

Group Member	Contribution
Deniz Çınar Bayburt	His main contribution was the implementation and assessment of the baseline classifiers. This included developing the one-vs-rest logistic regression framework, integrating lasso and ridge regularization into the training process, and performing model comparison under cross-validation. The contribution also covered plotting and interpreting performance trends such as loss vs iteration and accuracy vs lambda, a heatmap, which helped evaluate convergence and regularization effects.
Osman Gök	His main contribution was designing the preprocessing and feature engineering workflow. He transformed the raw league data into chronological team histories, generated aggregated features such as recent attacking, defensive, and disciplinary performance, and prepared the normalized input matrices used in the regression models. He also helped reduce feature dependency by examining correlation structure and model coefficients, which improved the interpretability of the final feature space.

Future Work

The average accuracies are 52.84%, 52.87%, and 52.9% for lasso-regularized logistic regression, ridge-regularized logistic regression, and standard logistic regression, respectively. These results are a baseline for our future work. The focus of the next step will be implementing our proposed XGBoost and Multi-Layer Perceptrons (MLP) algorithms to improve and surpass the output result. We also plan to leverage the existing feature set in a non-linear model to capture more complex models and create more detailed, advanced features, such as xG(expected goals for), NPxG (expected goals for without penalties), PPDA(passes allowed per defensive action in opposition half), and DC(passes completed within an estimated 20 meters of goal).

References

[1] DataHub, "English Premier League," *DataHub.io*, 2026. [Online]. Available: https://datahub.io/core/english-premier-league. [Accessed: March. 23, 2026].

Appendix

Appendix A)

LogisticRegression

```
from read_data import PremierLeagueDataProcessor
import math
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns



data = PremierLeagueDataProcessor(start=True)
```

```
X_map = data.X
X = data.X_norm
y_all = data.y
y_folds = data.y_splits
y_zero_vs_rest = data.Y_zero_vs_rest
y_one_vs_rest = data.Y_one_vs_rest
y_two_vs_rest = data.Y_two_vs_rest
beta_init = data.beta
#print(len(beta))
"""print((X_train.iloc[0]))"""


def sigmoid(x):


    #print(1.0 / (1.0 + np.exp(-x)))
    return 1.0 / (1.0 + np.exp(-x))

def soft_threshold(rho, gamma):
    if rho > gamma:
        return rho - gamma
    elif rho < -gamma:
        return rho + gamma
    else:
        return 0


def sigmoid_matrix(z):
    z = np.clip(z, -250, 250)
    return 1.0 / (1.0 + np.exp(-z))


def binary_logistic_loss(X, y, beta, regularizer="none", lmbda=0.0,
eps=1e-12):
    X_mat = X.to_numpy(dtype=float) if hasattr(X, "to_numpy") else
np.asarray(X, dtype=float)
    y_arr = np.asarray(y, dtype=float).reshape(-1)
    beta_arr = np.asarray(beta, dtype=float).reshape(-1)

    probs = sigmoid_matrix(X_mat @ beta_arr)
    probs = np.clip(probs, eps, 1.0 - eps)

    base_loss = -np.mean(y_arr * np.log(probs) + (1.0 - y_arr) *
np.log(1.0 - probs))
```

```
    if regularizer == "lasso":
        penalty = lmbda * np.sum(np.abs(beta_arr[1:]))
    elif regularizer == "ridge":
        penalty = 0.5 * lmbda * np.sum(beta_arr[1:] ** 2)
    else:
        penalty = 0.0

    return float(base_loss + penalty)



def ridge_logistic_regression(
    X_folds,
    y_folds,
    beta_init,
    lmbda=0.1,
    learning_rate=0.01,
    iterations=1000,
    tol=1e-6,
    return_histories=False
):

    num_folds = len(X_folds)


    beta_init = np.asarray(beta_init, dtype=float).reshape(-1)
    betas = []
    loss_histories = []

    for leave_out in range(num_folds):
        X_train = np.concatenate([
            X_folds[i].to_numpy(dtype=float) if hasattr(X_folds[i],
"to_numpy") else np.asarray(X_folds[i], dtype=float)
            for i in range(num_folds) if i != leave_out
        ], axis=0)

        y_train = np.concatenate([
            np.asarray(y_folds[i], dtype=float).reshape(-1)
            for i in range(num_folds) if i != leave_out
        ], axis=0)

        n_samples, n_features = X_train.shape
```

```
        beta = beta_init.copy()
        fold_losses = []


        if return_histories:
            fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="ridge", lmbda=lmbda))
        for _ in range(iterations):
            linear_pred = X_train @ beta
            predictions = sigmoid_matrix(linear_pred)

            error = predictions - y_train
            gradient = (X_train.T @ error) / n_samples

            penalty = lmbda * beta
            penalty[0] = 0.0
            gradient += penalty

            beta_new = beta - learning_rate * gradient

            if np.linalg.norm(beta_new - beta) < tol:
                beta = beta_new
                fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="ridge", lmbda=lmbda))
                break

            beta = beta_new
            if return_histories:
                fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="ridge", lmbda=lmbda))

        betas.append(beta.copy())
        if return_histories:
            loss_histories.append(fold_losses)

    if return_histories:
        return betas, loss_histories
    return betas

def standard_logistic_regression(
    X_folds,
    y_folds,
    beta_init,
    learning_rate=0.01,
    iterations=1000,
```

```
    tol=1e-6,
    return_histories=False
):

    num_folds = len(X_folds)


    beta_init = np.asarray(beta_init, dtype=float).reshape(-1)
    betas = []
    loss_histories = []

    for leave_out in range(num_folds):
        X_train = np.concatenate([
            X_folds[i].to_numpy(dtype=float) if hasattr(X_folds[i],
"to_numpy") else np.asarray(X_folds[i], dtype=float)
            for i in range(num_folds) if i != leave_out
        ], axis=0)

        y_train = np.concatenate([
            np.asarray(y_folds[i], dtype=float).reshape(-1)
            for i in range(num_folds) if i != leave_out
        ], axis=0)

        n_samples, n_features = X_train.shape



        beta = beta_init.copy()
        fold_losses = []
        if return_histories:
            fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="none", lmbda=0.0))
        for _ in range(iterations):
            linear_pred = X_train @ beta
            predictions = sigmoid_matrix(linear_pred)

            error = predictions - y_train
            gradient = (X_train.T @ error) / n_samples

            beta_new = beta - learning_rate * gradient

            if np.linalg.norm(beta_new - beta) < tol:
                beta = beta_new
                if return_histories:
```

```
                    fold_losses.append(binary_logistic_loss(X_train,
y_train, beta, regularizer="none", lmbda=0.0))
                break

            beta = beta_new
            if return_histories:
                fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="none", lmbda=0.0))

        betas.append(beta.copy())
        if return_histories:
            loss_histories.append(fold_losses)

    if return_histories:
        return betas, loss_histories
    return betas


def lasso_logistic_regression(
    X_input,
    y_input,
    beta_init,
    lmbda,
    iterations=500,
    tol=1e-9,
    num_folds=8,
    return_histories=False
):

    if isinstance(X_input, (list, tuple)):


        X_folds = [
            x.to_numpy(dtype=float) if hasattr(x, "to_numpy") else
np.asarray(x, dtype=float)
            for x in X_input
        ]
        y_folds = [
            np.asarray(y, dtype=float).reshape(-1)
            for y in y_input
        ]
    else:
        X_array = X_input.to_numpy(dtype=float) if hasattr(X_input,
"to_numpy") else np.asarray(X_input, dtype=float)
        y_array = np.asarray(y_input, dtype=float).reshape(-1)
```

```
        X_folds = np.array_split(X_array, num_folds, axis=0)
        y_folds = np.array_split(y_array, num_folds, axis=0)

    beta_init = np.asarray(beta_init, dtype=float).reshape(-1)
    p = X_folds[0].shape[1]



    betas = []
    loss_histories = []

    for leave_out in range(num_folds):
        train_ix = [k for k in range(num_folds) if k != leave_out]

        X_train = np.concatenate([X_folds[k] for k in train_ix], axis=0)
        y_train = np.concatenate([y_folds[k] for k in train_ix],
axis=0).reshape(-1)

        n, p_train = X_train.shape


        beta = beta_init.copy()
        fold_losses = []
        if return_histories:
            fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="lasso", lmbda=lmbda))

        for _ in range(iterations):
            beta_old = beta.copy()

            linear_preds = X_train @ beta
            probs = 1.0 / (1.0 + np.exp(-linear_preds))
            w = np.maximum(probs * (1.0 - probs), 1e-8)
            z = linear_preds + (y_train - probs) / w

            for j in range(p):
                gamma = 0.0 if j == 0 else n * lmbda

                partial = linear_preds - X_train[:, j] * beta[j]
                rj = np.sum((w * X_train[:, j]) * (z - partial))
                denom = np.sum(w * (X_train[:, j] ** 2)) + 1e-12
```

```
                if j == 0:
                    beta_j_new = rj / denom
                else:
                    beta_j_new = np.sign(rj) * max(abs(rj) - gamma, 0.0) /
denom

                delta = beta_j_new - beta[j]
                if delta != 0.0:
                    linear_preds += X_train[:, j] * delta
                    beta[j] = beta_j_new

            if np.linalg.norm(beta - beta_old) < tol:
                if return_histories:
                    fold_losses.append(binary_logistic_loss(X_train,
y_train, beta, regularizer="lasso", lmbda=lmbda))
                break

            if return_histories:
                fold_losses.append(binary_logistic_loss(X_train, y_train,
beta, regularizer="lasso", lmbda=lmbda))

        betas.append(beta.copy())
        if return_histories:
            loss_histories.append(fold_losses)

    if return_histories:
        return betas, loss_histories
    return betas


def test(X, beta0,beta1, beta2,y):
    X_mat = X.to_numpy(dtype=float) if hasattr(X, 'to_numpy') else
np.asarray(X, dtype=float)
    if X_mat.ndim == 1:
        X_mat = X_mat.reshape(1, -1)

    y_arr = np.asarray(y).reshape(-1).astype(int)


    result_0 = sigmoid_matrix(X_mat @ beta0)
    result_1 = sigmoid_matrix(X_mat @ beta1)
    result_2 = sigmoid_matrix(X_mat @ beta2)
    predictions = np.argmax(np.column_stack([result_0, result_1,
result_2]), axis=1)
    return float(np.mean(predictions == y_arr))
```

```
def evaluate_fold_models(X_folds, y_folds, beta0_list, beta1_list,
beta2_list):
    num_folds = len(X_folds)


    success_rates = []
    for fold_idx in range(num_folds):
        fold_score = test(
            X_folds[fold_idx],
            beta0_list[fold_idx],
            beta1_list[fold_idx],
            beta2_list[fold_idx],
            y_folds[fold_idx]
        )
        success_rates.append(fold_score)
    return success_rates


def t_critical_95(df):
    t_table = {
        1: 12.706, 2: 4.303, 3: 3.182, 4: 2.776, 5: 2.571,
        6: 2.447, 7: 2.365, 8: 2.306, 9: 2.262, 10: 2.228,
        11: 2.201, 12: 2.179, 13: 2.160, 14: 2.145, 15: 2.131,
        16: 2.120, 17: 2.110, 18: 2.101, 19: 2.093, 20: 2.086,
        21: 2.080, 22: 2.074, 23: 2.069, 24: 2.064, 25: 2.060,
        26: 2.056, 27: 2.052, 28: 2.048, 29: 2.045, 30: 2.042
    }
    if df <= 0:
        return 0.0
    if df in t_table:
        return t_table[df]
    return 1.96


def confidence_interval(values, confidence=0.95):
    values = np.asarray(values, dtype=float)
    n = len(values)
    mean = float(np.mean(values))

    if n < 2:
        return {
            "mean": mean,
            "std": 0.0,
```

```
            "sem": 0.0,
            "half_width": 0.0,
            "lower": mean,
            "upper": mean,
            "confidence": confidence,
            "n": n,
        }

    std = float(np.std(values, ddof=1))
    sem = std / np.sqrt(n)
    if np.isclose(confidence, 0.95):
        critical = t_critical_95(n - 1)
    else:
        critical = 1.96

    half_width = critical * sem
    return {
        "mean": mean,
        "std": std,
        "sem": sem,
        "half_width": half_width,
        "lower": mean - half_width,
        "upper": mean + half_width,
        "confidence": confidence,
        "n": n,
    }


def print_confidence_intervals(model_scores, confidence=0.95):
    confidence_pct = int(confidence * 100)
    print(f"\n=== {confidence_pct}% CONFIDENCE INTERVALS ===")
    for model_name, scores in model_scores.items():
        stats = confidence_interval(scores, confidence=confidence)
        print(
            f"{model_name}: mean={stats['mean']:.4f}, "
            f"CI=[{stats['lower']:.4f}, {stats['upper']:.4f}], "
            f"half-width={stats['half_width']:.4f}, n={stats['n']}"
        )


def plot_confidence_intervals(model_scores, confidence=0.95,
save_path="confidence_intervals.png", show_plot=True):
    confidence_pct = int(confidence * 100)
    model_names = list(model_scores.keys())
```

```
    stats_list = [confidence_interval(model_scores[name],
confidence=confidence) for name in model_names]
    means = [item["mean"] for item in stats_list]
    half_widths = [item["half_width"] for item in stats_list]

    x = np.arange(len(model_names))
    colors = ["#d95f02", "#1b9e77", "#7570b3"]

    plt.figure(figsize=(9, 6))
    plt.bar(x, means, yerr=half_widths, capsize=10,
color=colors[:len(model_names)], alpha=0.8)

    for idx, name in enumerate(model_names):
        fold_scores = np.asarray(model_scores[name], dtype=float)
        jitter = np.linspace(-0.08, 0.08, len(fold_scores)) if
len(fold_scores) > 1 else np.array([0.0])
        plt.scatter(np.full(len(fold_scores), idx) + jitter, fold_scores,
color="black", s=35, zorder=3)

    plt.xticks(x, model_names)
    plt.ylabel("Accuracy")
    plt.ylim(0.0, 1.0)
    plt.title(f"Model Accuracy with {confidence_pct}% Confidence
Intervals")
    plt.grid(axis="y", alpha=0.25)
    plt.tight_layout()
    plt.savefig(save_path, dpi=200, bbox_inches="tight")
    if show_plot:
        plt.show()
    plt.close()


def train_ovr_regression(regression_name, X_folds, y_zero_folds,
y_one_folds, y_two_folds, beta_init, lmbda=0.0):
    if regression_name == "lasso":
        beta0 = lasso_logistic_regression(X_folds, y_zero_folds,
beta_init, lmbda=lmbda)
        beta1 = lasso_logistic_regression(X_folds, y_one_folds, beta_init,
lmbda=lmbda)
        beta2 = lasso_logistic_regression(X_folds, y_two_folds, beta_init,
lmbda=lmbda)
    elif regression_name == "ridge":
        beta0 = ridge_logistic_regression(X_folds, y_zero_folds,
beta_init, lmbda=lmbda)
```

```
        beta1 = ridge_logistic_regression(X_folds, y_one_folds, beta_init,
lmbda=lmbda)
        beta2 = ridge_logistic_regression(X_folds, y_two_folds, beta_init,
lmbda=lmbda)
    elif regression_name == "standard":
        beta0 = standard_logistic_regression(X_folds, y_zero_folds,
beta_init)
        beta1 = standard_logistic_regression(X_folds, y_one_folds,
beta_init)
        beta2 = standard_logistic_regression(X_folds, y_two_folds,
beta_init)
    else:
        raise ValueError(f"Unknown regression name: {regression_name}")

    return beta0, beta1, beta2


def mean_accuracy_for_model(regression_name, X_folds, y_folds,
y_zero_folds, y_one_folds, y_two_folds, beta_init, lmbda=0.0):
    beta0, beta1, beta2 = train_ovr_regression(
        regression_name,
        X_folds,
        y_zero_folds,
        y_one_folds,
        y_two_folds,
        beta_init,
        lmbda=lmbda
    )
    fold_scores = evaluate_fold_models(X_folds, y_folds, beta0, beta1,
beta2)
    return float(np.mean(fold_scores)), fold_scores


def lambda_sweep_accuracy(X_folds, y_folds, y_zero_folds, y_one_folds,
y_two_folds, beta_init, lambda_values):
    lasso_scores = []
    ridge_scores = []

    standard_mean, standard_fold_scores = mean_accuracy_for_model(
        "standard", X_folds, y_folds, y_zero_folds, y_one_folds,
y_two_folds, beta_init
    )
    standard_scores = [standard_mean] * len(lambda_values)

    for idx, lmbda in enumerate(lambda_values):
```

```
        lasso_mean, _ = mean_accuracy_for_model(
            "lasso", X_folds, y_folds, y_zero_folds, y_one_folds,
y_two_folds, beta_init, lmbda=lmbda
        )
        ridge_mean, _ = mean_accuracy_for_model(
            "ridge", X_folds, y_folds, y_zero_folds, y_one_folds,
y_two_folds, beta_init, lmbda=lmbda
        )
        lasso_scores.append(lasso_mean)
        ridge_scores.append(ridge_mean)


    return {
        "lambda_values": np.asarray(lambda_values, dtype=float),
        "lasso": np.asarray(lasso_scores, dtype=float),
        "ridge": np.asarray(ridge_scores, dtype=float),
        "standard": np.asarray(standard_scores, dtype=float),
        "standard_fold_scores": np.asarray(standard_fold_scores,
dtype=float),
    }


def plot_lambda_accuracy_curves(sweep_results,
save_path="lambda_accuracy_curve.png", show_plot=True):
    lambda_values = sweep_results["lambda_values"]
    lasso_scores = sweep_results["lasso"]
    ridge_scores = sweep_results["ridge"]
    standard_scores = sweep_results["standard"]

    plt.figure(figsize=(10, 6))
    plt.plot(lambda_values, lasso_scores, label="Lasso Logistic",
linewidth=2.2, color="#d95f02")
    plt.plot(lambda_values, ridge_scores, label="Ridge Logistic",
linewidth=2.2, color="#1b9e77")
    plt.plot(lambda_values, standard_scores, label="Standard Logistic",
linewidth=2.2, linestyle="--", color="#7570b3")

    plt.xlabel("Lambda")
    plt.ylabel("Accuracy")
    plt.title("Lambda vs Accuracy")
    plt.ylim(0.0, 1.0)
    plt.grid(alpha=0.25)
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=200, bbox_inches="tight")
```

```
    if show_plot:
        plt.show()
    plt.close()


def pad_loss_history(loss_values, target_length):
    loss_values = list(loss_values)
    if len(loss_values) == 0:
        return np.full(target_length, np.nan, dtype=float)
    if len(loss_values) >= target_length:
        return np.asarray(loss_values[:target_length], dtype=float)
    padded = loss_values + [loss_values[-1]] * (target_length -
len(loss_values))
    return np.asarray(padded, dtype=float)


def mean_ovr_loss_curve(class_histories_for_fold):
    max_len = max(len(history) for history in class_histories_for_fold)
    padded = np.vstack([pad_loss_history(history, max_len) for history in
class_histories_for_fold])
    return np.nanmean(padded, axis=0)


def build_loss_history_summary(X_folds, y_zero_folds, y_one_folds,
y_two_folds, beta_init, fold_idx=0):
    _, lasso_h0 = lasso_logistic_regression(X_folds, y_zero_folds,
beta_init, lmbda=0.001, return_histories=True)
    _, lasso_h1 = lasso_logistic_regression(X_folds, y_one_folds,
beta_init, lmbda=0.01, return_histories=True)
    _, lasso_h2 = lasso_logistic_regression(X_folds, y_two_folds,
beta_init, lmbda=0.001, return_histories=True)

    _, ridge_h0 = ridge_logistic_regression(X_folds, y_zero_folds,
beta_init, lmbda=0.001, return_histories=True)
    _, ridge_h1 = ridge_logistic_regression(X_folds, y_one_folds,
beta_init, lmbda=0.01, return_histories=True)
    _, ridge_h2 = ridge_logistic_regression(X_folds, y_two_folds,
beta_init, lmbda=0.001, return_histories=True)

    _, standard_h0 = standard_logistic_regression(X_folds, y_zero_folds,
beta_init, return_histories=True)
    _, standard_h1 = standard_logistic_regression(X_folds, y_one_folds,
beta_init, return_histories=True)
    _, standard_h2 = standard_logistic_regression(X_folds, y_two_folds,
beta_init, return_histories=True)
```

```
    return {
        "Lasso": mean_ovr_loss_curve([lasso_h0[fold_idx],
lasso_h1[fold_idx], lasso_h2[fold_idx]]),
        "Ridge": mean_ovr_loss_curve([ridge_h0[fold_idx],
ridge_h1[fold_idx], ridge_h2[fold_idx]]),
        "Standard": mean_ovr_loss_curve([standard_h0[fold_idx],
standard_h1[fold_idx], standard_h2[fold_idx]]),
    }


def print_loss_history_summary(loss_history_summary, fold_idx=0):
    print(f"\n=== LOSS VS ITERATION SUMMARY (Fold {fold_idx}) ===")
    for model_name, curve in loss_history_summary.items():
        print(
            f"{model_name}: iterations={len(curve)}, "
            f"initial_loss={curve[0]:.6f}, final_loss={curve[-1]:.6f}"
        )


def plot_loss_vs_iteration(loss_history_summary, fold_idx=0,
save_path="loss_vs_iteration.png", show_plot=True):
    plt.figure(figsize=(10, 6))
    colors = {"Lasso": "#d95f02", "Ridge": "#1b9e77", "Standard":
"#7570b3"}

    for model_name, curve in loss_history_summary.items():
        iterations = np.arange(1, len(curve) + 1)
        plt.plot(iterations, curve, label=model_name, linewidth=2.2,
color=colors.get(model_name))

    plt.xlabel("Iteration")
    plt.ylabel("Loss")
    plt.title(f"Loss vs Iteration (Average OvR Loss, Fold {fold_idx})")
    plt.grid(alpha=0.25)
    plt.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=200, bbox_inches="tight")
    if show_plot:
        plt.show()
    plt.close()


def correlation_map(X):
```

```
    X_mat = X.to_numpy() if hasattr(X, 'to_numpy') else np.array(X,
dtype=float)


    X_sub = X_mat[:, 1:]


    average = np.mean(X_sub, axis=0)
    std = np.std(X_sub, axis=0, ddof=1)


    cov = np.cov(X_sub, rowvar=False, ddof=1)

    return average, std, cov

def plot_covariance_heatmap(X, labels=None, figsize=(14, 12),
cmap='coolwarm', annot=False):

    if plt is None or sns is None:
        raise ImportError("plot_covariance_heatmap için matplotlib ve
seaborn gerekli")

    _, _, cov = correlation_map(X)

    if labels is None:
        p = cov.shape[0]
        labels = [f"x{i}" for i in range(p)]

    plt.figure(figsize=figsize)
    sns.heatmap(cov, xticklabels=labels, yticklabels=labels,
                cmap=cmap, center=0, annot=annot, fmt=".2f",
                cbar_kws={"shrink": 0.8})
    plt.title("Covariance Matrix Heatmap (Bessel ddof=1)")
    plt.tight_layout()
    plt.show()




beta_zero_vs_rest_lasso = lasso_logistic_regression(X, y_zero_vs_rest,
beta_init, lmbda=0.001)
beta_one_vs_rest_lasso = lasso_logistic_regression(X, y_one_vs_rest,
beta_init, lmbda=0.01)
```

```
beta_two_vs_rest_lasso = lasso_logistic_regression(X, y_two_vs_rest,
beta_init, lmbda=0.001)

succes_for_lasso = evaluate_fold_models(
    X, y_folds,
    beta_zero_vs_rest_lasso,
    beta_one_vs_rest_lasso,
    beta_two_vs_rest_lasso
)

beta_zero_vs_rest_standard = standard_logistic_regression(X,
y_zero_vs_rest, beta_init)
beta_one_vs_rest_standard = standard_logistic_regression(X, y_one_vs_rest,
beta_init)
beta_two_vs_rest_standard = standard_logistic_regression(X, y_two_vs_rest,
beta_init)

succes_for_standard = evaluate_fold_models(
    X, y_folds,
    beta_zero_vs_rest_standard,
    beta_one_vs_rest_standard,
    beta_two_vs_rest_standard
)

beta_zero_vs_rest_ridge = ridge_logistic_regression(X, y_zero_vs_rest,
beta_init, lmbda=0.001)
beta_one_vs_rest_ridge = ridge_logistic_regression(X, y_one_vs_rest,
beta_init, lmbda=0.01)
beta_two_vs_rest_ridge = ridge_logistic_regression(X, y_two_vs_rest,
beta_init, lmbda=0.001)

succes_for_ridge = evaluate_fold_models(
    X, y_folds,
    beta_zero_vs_rest_ridge,
    beta_one_vs_rest_ridge,
    beta_two_vs_rest_ridge
)

for i in range(len(succes_for_lasso)):
    print(
        f"Fold {i}: "
        f"Lasso Logit: {succes_for_lasso[i]:.4f}, "
        f"Ridge Logit: {succes_for_ridge[i]:.4f}, "
        f"Standard Logit: {succes_for_standard[i]:.4f}"
    )
```

```
print(
    f"\nAverage Accuracy -> "
    f"Lasso: {np.mean(succes_for_lasso):.4f}, "
    f"Ridge: {np.mean(succes_for_ridge):.4f}, "
    f"Standard: {np.mean(succes_for_standard):.4f}"
)

model_scores = {
    "Lasso": succes_for_lasso,
    "Ridge": succes_for_ridge,
    "Standard": succes_for_standard,
}

print_confidence_intervals(model_scores, confidence=0.95)
plot_confidence_intervals(model_scores, confidence=0.95)

selected_fold_for_loss_plot = 0
loss_history_summary = build_loss_history_summary(
    X,
    y_zero_vs_rest,
    y_one_vs_rest,
    y_two_vs_rest,
    beta_init,
    fold_idx=selected_fold_for_loss_plot
)
print_loss_history_summary(loss_history_summary,
fold_idx=selected_fold_for_loss_plot)
plot_loss_vs_iteration(
    loss_history_summary,
    fold_idx=selected_fold_for_loss_plot,
    save_path="loss_vs_iteration.png",
    show_plot=True
)

max_loss_len = max(len(curve) for curve in loss_history_summary.values())
loss_curve_df = np.column_stack([
    np.arange(1, max_loss_len + 1),
    pad_loss_history(loss_history_summary["Lasso"], max_loss_len),
    pad_loss_history(loss_history_summary["Ridge"], max_loss_len),
    pad_loss_history(loss_history_summary["Standard"], max_loss_len),
])
np.savetxt(
    "loss_vs_iteration.csv",
    loss_curve_df,
```

```
    delimiter=",",
    header="iteration,lasso_loss,ridge_loss,standard_loss",
    comments=""
)

lambda_values = np.linspace(0.00, 4, 100)

sweep_results = lambda_sweep_accuracy(
    X,
    y_folds,
    y_zero_vs_rest,
    y_one_vs_rest,
    y_two_vs_rest,
    beta_init,
    lambda_values
)

best_lasso_idx = int(np.argmax(sweep_results["lasso"]))
best_ridge_idx = int(np.argmax(sweep_results["ridge"]))



lambda_curve_df = np.column_stack([
    sweep_results["lambda_values"],
    sweep_results["lasso"],
    sweep_results["ridge"],
    sweep_results["standard"],
])
np.savetxt(
    "lambda_accuracy_curve.csv",
    lambda_curve_df,
    delimiter=",",
    header="lambda,lasso_accuracy,ridge_accuracy,standard_accuracy",
    comments=""
)

plot_lambda_accuracy_curves(sweep_results)
```

Appendix B)

read_data.py

```
import pandas as pd
import numpy as np
import time

"""
##########################################################################
##########################
For the prediction of the match that is not played yet, the statistics of
that match cannot be used.
Therefore, we planned to modelize our input vector X from the statistics
of previous matches. For example,
we cannot plug in the shot on target of the teams from the non-played
match, instead, we model three main features for each of the teams:
(In total we 23x2+ 1 = 47 features for each match)
Each team have different dataset for home and away matches. For example,
Chelsea has two dataset: one for home matches and one for away matches.
1) Attack srength:
    - gf_last15_avg: It is calculated by the total goals scored by the
team in the last 15 matches before that match.
    - gf_last5_avg: It is calculated by the total goals scored by the team
in the last 5 matches before that match.
    - shot_on_target_last15_avg: It is calculated by the total shots on
target of the team in the last 15 matches before that match.
    - shot_on_target_last5_avg: It is calculated by the total shots on
target of the team in the last 5 matches before that match.
    - shot_last15_avg: It is calculated by the total shots of the team in
the last 15 matches before that match.
    - shot_last5_avg: It is calculated by the total shots of the team in
the last 5 matches before that match.
    - corner_last15_avg: It is calculated by the total corners of the team
in the last 15 matches before that match.
    - corner_last5_avg: It is calculated by the total corners of the team
in the last 5 matches before that match.
    - faul_againts_last15_avg: It is calculated by the total fauls of the
OPPENENTS in the last 15 matches before that match.
    - faul_againts_last5_avg: It is calculated by the total fauls of the
OPPENENTS in the last 5 matches before that match.

2) Defence strength:

    - ga_last5_avg: It is calculated by the total goals conceded by the
team in the last 5 matches before that match.
```

- zga_last5_avg: It is calculated by the total goals conceded by the team in the last 15 matches before that match.
- ga_last15_avg: It is calculated by the total goals conceded by the team in the last 15 matches before that match.
- shot_on_target_against_last5_avg: It is calculated by the total shots on target against the team in the last 5 matches before that match.
- shot_on_target_against_last15_avg: It is calculated by the total shots on target against the team in the last 15 matches before that match.
- shot_against_last5_avg: It is calculated by the total shots against the team in the last 5 matches before that match.
- shot_against_last15_avg: It is calculated by the total shots against the team in the last 15 matches before that match.
- faul_last5_avg: It is calculated by the total fauls of the team in the last 5 matches before that match.
- faul_last15_avg: It is calculated by the total fauls of the team in the last 15 matches before that match.

3) Discipline:
- card_points_last15_avg: It is calculated by the total card points of the team in the last 15 matches before that match.
- card_points_last5_avg: It is calculated by the total card points of the team in the last 5 matches before that match.
- cord_points_per_faul_last15_avg: It is calculated by the total card points per faul of the team in the last 15 matches before that match.
- cord_points_per_faul_last5_avg: It is calculated by the total card points per faul of the team in the last 5 matches before that match.

The last X vecotor = [1, Away_team_attack_strength, Away_team_defence_strength, Away_team_discipline, Home_team_attack_strength, Home_team_defence_strength, Home_team_discipline]

Data leakage:
In the process of creating the input vector X, we have to be careful about data beacuse some season some of the set of the teams cannot be played due to bad result of the previous reason. Therefore, some teams played less match than others. While taking the average of the last 5 matches, some of the teams haven't played the match in PL for a few seasons from the day of the match.
Therefore, we arranged the data and put zero for the matches that they haven't played. For example, if a team has played only 3 matches before the match that we want to predict, we take the average of those 3 matches and put zero for the rest of the 2 matches.
In this way, we can avoid data leakage and make our model more robust.

Independency:

```
In the theoretical world, the input vectors dimensions are assumed to be
independent. However, in the real world, there are some correlations
between the features. For example, the attack strength and defence
strength of a team are correlated. Therefore, we have to be
careful about the correlations between the features and try to minimize
them as much as possible. Lasso?


##########################################################################
############################
"""


class PremierLeagueDataProcessor:
    def __init__(self, start = True):

        self.data_array = []
        self.data_array_standings_last = [[],[],[],[],[],[],[],[],[],[]]
        self.data_array_standings = []
        self.score = []
        self.teams = []
        self.teams_rates_all = []
        self.teams_statistics = []
        self.teams_away_statistics_pure = []
        self.teams_home_statistics_pure = []
        self.yellow_cards = []
        self.gecici = []
        self.arranged_data_home = []
        self.arranged_data_away = []
        self.max_games = 0
        self.data_frames = []
        self.all_matches_df = None
        self.dataset_df = None
        self.X = None
        self.y_raw = None
        self.y_int = None
        self.Y_one_hot = None
        self.cross_validation = 8
        self.X_train_df = None
        self.X_test_df =None
        self.y_train_df = None
        self.y_test_df =None
        self.Y_zero_vs_rest = []
        self.Y_one_vs_rest =[]
        self.Y_two_vs_rest = []
```

```
        self.beta = []
        self.X_train = [0]*self.cross_validation
        self.X_test = [0]*self.cross_validation
        self.y_train = [0]*self.cross_validation
        self.y_test = [0]*self.cross_validation

        self.start = start
        self.base_feature_names = [
            'gf_last15_avg', 'gf_last5_avg',
            'shot_on_target_last15_avg', 'shot_on_target_last5_avg',
            'shot_last15_avg', 'shot_last5_avg',
            'corner_last15_avg', 'corner_last5_avg',
            'faul_againts_last15_avg', 'faul_againts_last5_avg',
            'ga_last5_avg', 'zga_last5_avg', 'ga_last15_avg',
            'shot_on_target_against_last5_avg',
'shot_on_target_against_last15_avg',
            'shot_against_last5_avg', 'shot_against_last15_avg',
            'faul_last5_avg', 'faul_last15_avg',
            'card_points_last15_avg', 'card_points_last5_avg',
            'cord_points_per_faul_last15_avg',
'cord_points_per_faul_last5_avg'
        ]
        self.home_feature_names = [f'home_{name}' for name in
self.base_feature_names]
        self.away_feature_names = [f'away_{name}' for name in
self.base_feature_names]
        self.feature_names = self.home_feature_names +
self.away_feature_names
        if self.start == True:
            self.run_all()
    def _parse_date_series(self, s):
        out = pd.to_datetime(s, format="%d/%m/%Y", errors="coerce")
        mask = out.isna()
        if mask.any():
            out2 = pd.to_datetime(s[mask], format="%d/%m/%y",
errors="coerce")
            out.loc[mask] = out2
        return out

    def _find_entry(self, collection, team):
        for item in collection:
            if item['Team'] == team:
                return item
        raise ValueError(f"Team not found: {team}")
```

```
    def _resolve_team(self, team_name):
        normalized = team_name.strip().lower()
        for team in self.teams:
            if team.lower() == normalized:
                return team
        raise ValueError(f"Unknown team: {team_name}")

    def _one_hot(self, y, num_classes=3):
        out = np.zeros((len(y), num_classes), dtype=float)
        for idx, label in enumerate(y):
            out[idx, int(label)] = 1.0
        return out

    def _label_to_int(self, ftr, fthg, ftag):
        if isinstance(ftr, str):
            ftr = ftr.strip().upper()
            if ftr == 'H':
                return 0
            if ftr == 'D':
                return 1
            if ftr == 'A':
                return 2
        if float(fthg) > float(ftag):
            return 0
        if float(fthg) == float(ftag):
            return 1
        return 2

    def load_data(self):

        fifteen_sixteen = pd.read_csv('season-1516.csv')
        sixteen_seventeen = pd.read_csv('season-1617.csv')
        seventeen_eighteen = pd.read_csv('season-1718.csv')
        eighteen_nineteen = pd.read_csv('season-1819.csv')
        nineteen_twenty = pd.read_csv('season-1920.csv')
        twenty_twentyone = pd.read_csv('season-2021.csv')
        twentyone_twentytwo = pd.read_csv('season-2122.csv')
        twentytwo_twentythree = pd.read_csv('season-2223.csv')
        twentythree_twentyfour = pd.read_csv('season-2324.csv')
        twentyfour_twentyfive = pd.read_csv('season-2425.csv')

        self.data_array.append(fifteen_sixteen.to_numpy())
        self.data_array.append(sixteen_seventeen.to_numpy())
        self.data_array.append(seventeen_eighteen.to_numpy())
        self.data_array.append(eighteen_nineteen.to_numpy())
```

```
        self.data_array.append(nineteen_twenty.to_numpy())
        self.data_array.append(twenty_twentyone.to_numpy())
        self.data_array.append(twentyone_twentytwo.to_numpy())
        self.data_array.append(twentytwo_twentythree.to_numpy())
        self.data_array.append(twentythree_twentyfour.to_numpy())
        self.data_array.append(twentyfour_twentyfive.to_numpy())

        self.data_frames = [
            fifteen_sixteen,
            sixteen_seventeen,
            seventeen_eighteen,
            eighteen_nineteen,
            nineteen_twenty,
            twenty_twentyone,
            twentyone_twentytwo,
            twentytwo_twentythree,
            twentythree_twentyfour,
            twentyfour_twentyfive
        ]

        self.data_array = np.array(self.data_array, dtype=object)

        fifteen_sixteen_standings = pd.read_csv('1516_standings.csv')
        sixteen_seventeen_standings = pd.read_csv('1617_standings.csv')
        seventeen_eighteen_standings = pd.read_csv('1718_standings.csv')
        eighteen_nineteen_standings = pd.read_csv('1819_standings.csv')
        nineteen_twenty_standings = pd.read_csv('1920_standings.csv')
        twenty_twentyone_standings = pd.read_csv('2021_standings.csv')
        twentyone_twentytwo_standings = pd.read_csv('2122_standings.csv')
        twentytwo_twentythree_standings =
pd.read_csv('2223_standings.csv')
        twentythree_twentyfour_standings =
pd.read_csv('2324_standings.csv')
        twentyfour_twentyfive_standings =
pd.read_csv('2425_standings.csv')

        for i in range(0,10):
            if i == 0:
                a = fifteen_sixteen_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 1:
                a = sixteen_seventeen_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 2:
                a = seventeen_eighteen_standings.to_numpy()
```

```
                self.data_array_standings.append(a[1:])
            elif i == 3:
                a = eighteen_nineteen_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 4:
                a = nineteen_twenty_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 5:
                a = twenty_twentyone_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 6:
                a = twentyone_twentytwo_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 7:
                a = twentytwo_twentythree_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            elif i == 8:
                a = twentythree_twentyfour_standings.to_numpy()
                self.data_array_standings.append(a[1:])
            else:
                a = twentyfour_twentyfive_standings.to_numpy()
                self.data_array_standings.append(a[1:])

        self.data_array_standings = np.array(self.data_array_standings,
dtype=object)

        required_cols = [
            'Date', 'HomeTeam', 'AwayTeam', 'FTHG', 'FTAG', 'FTR',
            'HTHG', 'HTAG', 'HS', 'AS', 'HST', 'AST', 'HF', 'AF',
            'HC', 'AC', 'HY', 'AY', 'HR', 'AR'
        ]
        merged = []
        for season_idx, df in enumerate(self.data_frames):
            xdf = df[required_cols].copy()
            xdf['season_idx'] = season_idx
            xdf['row_idx'] = np.arange(len(xdf))
            xdf['parsed_date'] = self._parse_date_series(xdf['Date'])
            merged.append(xdf)

        self.all_matches_df = pd.concat(merged, ignore_index=True)
        self.all_matches_df['parsed_date_isna'] =
self.all_matches_df['parsed_date'].isna().astype(int)
        self.all_matches_df = self.all_matches_df.sort_values(
            by=['parsed_date_isna', 'parsed_date', 'season_idx',
'row_idx'],
```

```
            kind='mergesort'
        ).reset_index(drop=True)
        self.all_matches_df['home_side_season_idx'] =
self.all_matches_df.groupby(['season_idx', 'HomeTeam']).cumcount()
        self.all_matches_df['away_side_season_idx'] =
self.all_matches_df.groupby(['season_idx', 'AwayTeam']).cumcount()

    def calculate_scores_and_rates(self):
        score_fifteen_sixteen = 0

        for j in range(len(self.data_array)):
            x = 0
            for i in range(len(self.data_array[j])):
                if self.data_array[j][i][5] == 'H':
                    x += 3
                elif self.data_array[j][i][5] == 'A':
                    x += 3
                else:
                    x += 1
            self.score.append(x)

        self.score = np.array(self.score)

        Total_score = 0
        for i in range(len(self.score)):
            Total_score += self.score[i]

        """Home rate and Away rate may be evelauted using just their
types"""
        for j in range(len(self.data_array)):
            for i in range(len(self.data_array_standings[j])):
                x = {
                    'Team': self.data_array_standings[j][i][1],
                    'Score': int(self.data_array_standings[j][i][17]),
                    'Total_rate': int(self.data_array_standings[j][i][17])
/ self.score[j],
                    'Home_rate': (int(self.data_array_standings[j][i][3])
* 3 + int(self.data_array_standings[j][i][4]) * 1 )/ self.score[j],
                    'Away_rate': (int(self.data_array_standings[j][i][7])
* 3 + int(self.data_array_standings[j][i][8]) * 1)/ self.score[j]
                }
                self.data_array_standings_last[j].append(x)

        self.data_array_standings_last = np.array([np.array(season) for
season in self.data_array_standings_last], dtype=object)
```

```
        for data in self.data_array_standings_last:
            for i in data:
                team = i['Team']
                team = team.split(' (')[0]
                self.teams.append(team)

        self.teams = np.array(list(set(self.teams)))

        for i in self.teams:
            score_temp = 0
            total_rate = 0
            home_rate = 0
            away_rate = 0
            for j in range(len(self.data_array_standings_last)):
                for k in range(len(self.data_array_standings_last[j])):
                    if self.data_array_standings_last[j][k]['Team'] == i:
                        total_rate += self.data_array_standings_last[j][k]
['Total_rate']
                        home_rate += self.data_array_standings_last[j][k]
['Home_rate']
                        away_rate += self.data_array_standings_last[j][k]
['Away_rate']
            x = {'Team': i,
                 'Total_rate_all': total_rate,
                 'Home_rate_all': home_rate,
                 'Away_rate_all': away_rate}
            self.teams_rates_all.append(x)

        self.teams_rates_all = np.array(self.teams_rates_all,
dtype=object)

    def process_match_statistics(self):
        #-------------------------------------------------------
        """We classified away team and home team separetly. For example,
Chealse has two folder: away and home"""
        for team in self.teams:
            home_statistics = []
            away_statistics = []
            for season in range(len(self.data_array)):
                season_columns = self.data_frames[season].columns
                for match in self.data_array[season]:
                    row = dict(zip(season_columns, match))
                    if team == row['HomeTeam']:
```

```
                        a = {'FTHG': row['FTHG'],'HTHG': row['HTHG'],
'HS': row['HS'], 'HST': row['HST'], 'HF': row['HF'],'HC': row['HC'],'HY':
row['HY'],'HR': row['HR'], 'GA': row['FTAG'], 'HTGA': row['HTAG'], 'FA':
row['AF'], 'SA': row['AS'], 'STA': row['AST']}
                        home_statistics.append(a)
                    elif team == row['AwayTeam']:
                        a = {'FTAG': row['FTAG'],'HTAG': row['HTAG'],
'AS': row['AS'], 'AST': row['AST'], 'AF': row['AF'],'AC': row['AC'],'AY':
row['AY'],'AR': row['AR'], 'GA': row['FTHG'], 'HTGA': row['HTHG'], 'FA':
row['HF'], 'SA': row['HS'], 'STA': row['HST']}
                        away_statistics.append(a)

            x = {'Team': team, 'Statistics_away':
np.array(away_statistics), 'Statistics_home': np.array(home_statistics)}
            self.teams_statistics.append(x)

        self.teams_statistics = np.array(self.teams_statistics,
dtype=object)

        for team in self.teams:
            FTGAFH = []
            HTGAFH = []
            FTGAFA = []
            HTGAFA = []
            FTHG = []
            FTAG = []
            HTHG = []
            HTAG = []
            HS = []
            AS = []
            HST = []
            AST = []
            HF = []
            AF = []
            HC = []
            AC = []
            HY = []
            AY = []
            AR = []
            HR = []
            FAA, SAA, STAA = [], [], []
            FAH, SAH, STAH = [], [], []
            for i in self.teams_statistics:
                if i['Team'] == team:
                    for j in i['Statistics_away']:
```

```
                        FTAG.append(j['FTAG'])
                        HTAG.append(j['HTAG'])
                        AS.append(j['AS'])
                        AST.append(j['AST'])
                        AF.append(j['AF'])
                        AC.append(j['AC'])
                        AR.append(j['AR'])
                        AY.append(j['AY'])
                        FTGAFA.append(j['GA'])
                        HTGAFA.append(j['HTGA'])
                        FAA.append(j['FA'])
                        SAA.append(j['SA'])
                        STAA.append(j['STA'])
                    for k in i['Statistics_home']:
                        FTHG.append(k['FTHG'])
                        HTHG.append(k['HTHG'])
                        HS.append(k['HS'])
                        HST.append(k['HST'])
                        HF.append(k['HF'])
                        HC.append(k['HC'])
                        HY.append(k['HY'])
                        HR.append(k['HR'])
                        FTGAFH.append(k['GA'])
                        HTGAFH.append(k['HTGA'])
                        FAH.append(k['FA'])
                        SAH.append(k['SA'])
                        STAH.append(k['STA'])

            x = {'Team': team,
                 'FTAG': np.array(FTAG), 'HTAG': np.array(HTAG), 'AS':
np.array(AS),
                 'AST': np.array(AST), 'AF': np.array(AF), 'AC':
np.array(AC),
                 'AR': np.array(AR), 'AY': np.array(AY), 'GA':
np.array(FTGAFA), 'HTGA': np.array(HTGAFA), 'FA': np.array(FAA), 'SA':
np.array(SAA), 'STA': np.array(STAA) }

            y = {'Team': team,
                 'FTHG': np.array(FTHG), 'HTHG': np.array(HTHG), 'HS':
np.array(HS),
                 'HST': np.array(HST), 'HF': np.array(HF), 'HC':
np.array(HC),
                 'HY': np.array(HY), 'HR': np.array(HR), 'GA':
np.array(FTGAFH), 'HTGA': np.array(HTGAFH),'FA': np.array(FAH), 'SA':
np.array(SAH), 'STA': np.array(STAH)}
```

```
            self.teams_away_statistics_pure.append(x)
            self.teams_home_statistics_pure.append(y)

        self.teams_away_statistics_pure =
np.array(self.teams_away_statistics_pure, dtype=object)
        self.teams_home_statistics_pure =
np.array(self.teams_home_statistics_pure, dtype=object)

        for team in self.teams:
            for i in range(len(self.teams_away_statistics_pure)):
                if self.teams_away_statistics_pure[i]['Team'] == team:
                    if len(self.teams_away_statistics_pure[i]['AS']) >
self.max_games:
                        self.max_games =
len(self.teams_away_statistics_pure[i]['AS'])

    def handle_missing_seasons(self):
        for team in self.teams:
            season_number_array = []

            for season_number in range(len(self.data_array_standings)):
                it_is_not = 0
                for i in self.data_array_standings[season_number]:
                    if team != i[1]:
                        it_is_not = 1
                    else:
                        it_is_not = 0
                        break
                if it_is_not == 1:
                    season_number_array.append(season_number)
                else:
                    pass

            self.gecici.append({'Team': team, 'Season number':
np.array(season_number_array)})

        self.gecici = np.array(self.gecici, dtype=object)

        zero = np.zeros(19, dtype=int)

        for team in self.teams:
            for i in self.teams_away_statistics_pure:
                if i['Team'] == team:
                    for gecicix in self.gecici:
```

```
                        if gecicix['Team'] == team:
                            if len(gecicix['Season number']) > 0:
                                for j in range(len(gecicix['Season
number'])):
                                    if  gecicix['Season number'][j] != 9:
                                        i['AS'] = np.insert(i['AS'],
gecicix['Season number'][j]*19, zero)
                                        i['AST'] = np.insert(i['AST'],
gecicix['Season number'][j]*19, zero)
                                        i['AF'] = np.insert(i['AF'],
gecicix['Season number'][j]*19, zero)
                                        i['AC'] = np.insert(i['AC'],
gecicix['Season number'][j]*19, zero)
                                        i['AR'] = np.insert(i['AR'],
gecicix['Season number'][j]*19, zero)
                                        i['AY'] = np.insert(i['AY'],
gecicix['Season number'][j]*19, zero)
                                        i['GA'] = np.insert(i['GA'],
gecicix['Season number'][j]*19, zero)
                                        i['HTGA'] = np.insert(i['HTGA'],
gecicix['Season number'][j]*19, zero)
                                        i['FTAG'] = np.insert(i['FTAG'],
gecicix['Season number'][j]*19, zero)
                                        i['HTAG'] = np.insert(i['HTAG'],
gecicix['Season number'][j]*19, zero)
                                        i['FA'] = np.insert(i['FA'],
gecicix['Season number'][j]*19, zero)
                                        i['SA'] = np.insert(i['SA'],
gecicix['Season number'][j]*19, zero)
                                        i['STA'] = np.insert(i['STA'],
gecicix['Season number'][j]*19, zero)

                                    elif  gecicix['Season number'][j] ==
9:
                                        i['AS'] = np.append(i['AS'], zero)
                                        i['AST'] = np.append(i['AST'],
zero)
                                        i['AF'] = np.append(i['AF'], zero)
                                        i['AC'] = np.append(i['AC'], zero)
                                        i['AR'] = np.append(i['AR'], zero)
                                        i['AY'] = np.append(i['AY'], zero)
                                        i['GA'] = np.append(i['GA'], zero)
                                        i['HTGA'] = np.append(i['HTGA'],
zero)
```

```
                                        i['FTAG'] = np.append(i['FTAG'],
zero)
                                        i['HTAG'] = np.append(i['HTAG'],
zero)
                                        i['FA'] = np.append(i['FA'], zero)
                                        i['SA'] = np.append(i['SA'], zero)
                                        i['STA'] = np.append(i['STA'],
zero)

            for i in self.teams_home_statistics_pure:
                if i['Team'] == team:
                    for gecicix in self.gecici:
                        if gecicix['Team'] == team:
                            if len(gecicix['Season number']) > 0:
                                for j in range(len(gecicix['Season
number'])):
                                    if  gecicix['Season number'][j] != 9:
                                        i['HS'] = np.insert(i['HS'],
gecicix['Season number'][j]*19, zero)
                                        i['HST'] = np.insert(i['HST'],
gecicix['Season number'][j]*19, zero)
                                        i['HF'] = np.insert(i['HF'],
gecicix['Season number'][j]*19, zero)
                                        i['HC'] = np.insert(i['HC'],
gecicix['Season number'][j]*19, zero)
                                        i['HR'] = np.insert(i['HR'],
gecicix['Season number'][j]*19, zero)
                                        i['HY'] = np.insert(i['HY'],
gecicix['Season number'][j]*19, zero)
                                        i['GA'] = np.insert(i['GA'],
gecicix['Season number'][j]*19, zero)
                                        i['HTGA'] = np.insert(i['HTGA'],
gecicix['Season number'][j]*19, zero)
                                        i['FTHG'] = np.insert(i['FTHG'],
gecicix['Season number'][j]*19, zero)
                                        i['HTHG'] = np.insert(i['HTHG'],
gecicix['Season number'][j]*19, zero)
                                        i['FA'] = np.insert(i['FA'],
gecicix['Season number'][j]*19, zero)
                                        i['SA'] = np.insert(i['SA'],
gecicix['Season number'][j]*19, zero)
                                        i['STA'] = np.insert(i['STA'],
gecicix['Season number'][j]*19, zero)
```

```
                                    elif  gecicix['Season number'][j] ==
9:
                                        i['HS'] = np.append(i['HS'], zero)
                                        i['HST'] = np.append(i['HST'],
zero)
                                        i['HF'] = np.append(i['HF'], zero)
                                        i['HC'] = np.append(i['HC'], zero)
                                        i['HR'] = np.append(i['HR'], zero)
                                        i['HY'] = np.append(i['HY'], zero)
                                        i['GA'] = np.append(i['GA'], zero)
                                        i['HTGA'] = np.append(i['HTGA'],
zero)
                                        i['FTHG'] = np.append(i['FTHG'],
zero)
                                        i['HTHG'] = np.append(i['HTHG'],
zero)
                                        i['FA'] = np.append(i['FA'], zero)
                                        i['SA'] = np.append(i['SA'], zero)
                                        i['STA'] = np.append(i['STA'],
zero)
    def check_array_sizes(self):
        print("\n--- Home Statistics Array Sizes Check ---")
        for team_data in self.teams_home_statistics_pure:
            team_name = team_data['Team']
            print(f"\nTeam: {team_name}")

            for key, value in team_data.items():
                # 'Team' anahtarı bir string olduğu için onu atlıyoruz,
sadece array'lere bakıyoruz
                if isinstance(value, (np.ndarray, list)):
                    print(f"  {key}: {len(value)}")
                else:
                    print(f"  {key}: (Not an array/list:
{type(value).__name__})")
    def _history_window(self, arr, end_idx, window):
        hist = np.asarray(arr[max(0, end_idx - window):end_idx],
dtype=float)
        if len(hist) < window:
            hist = np.pad(hist, (window - len(hist), 0),
constant_values=0.0)
        return hist

    def _feature_row_from_side_stats(self, side_stats, side, end_idx):
        if side == 'away':
            gf = side_stats['FTAG']
```

```
            sot = side_stats['AST']
            shot = side_stats['AS']
            corner = side_stats['AC']
            foul_against = side_stats['FA']
            ga = side_stats['GA']
            sota = side_stats['STA']
            sa = side_stats['SA']
            foul = side_stats['AF']
            card_points = side_stats['AY'] + 2.0 * side_stats['AR']
        else:
            gf = side_stats['FTHG']
            sot = side_stats['HST']
            shot = side_stats['HS']
            corner = side_stats['HC']
            foul_against = side_stats['FA']
            ga = side_stats['GA']
            sota = side_stats['STA']
            sa = side_stats['SA']
            foul = side_stats['HF']
            card_points = side_stats['HY'] + 2.0 * side_stats['HR']

        def m(arr, window):
            return float(self._history_window(arr, end_idx,
window).mean())

        def z(arr, window):
            return float((self._history_window(arr, end_idx, window) ==
0).sum() / window)

        def r(num_arr, den_arr, window, eps=1e-8):
            num = self._history_window(num_arr, end_idx, window)
            den = self._history_window(den_arr, end_idx, window)
            return float(num.sum() / (den.sum() + eps))

        return np.array([
            m(gf, 15), m(gf, 5),
            m(sot, 15), m(sot, 5),
            m(shot, 15), m(shot, 5),
            m(corner, 15), m(corner, 5),
            m(foul_against, 15), m(foul_against, 5),
            m(ga, 5), z(ga, 5), m(ga, 15),
            m(sota, 5), m(sota, 15),
            m(sa, 5), m(sa, 15),
            m(foul, 5), m(foul, 15),
            m(card_points, 15), m(card_points, 5),
```

```
            r(card_points, foul, 15), r(card_points, foul, 5)
        ], dtype=float)

    def arrange_data(self):
        self.arranged_data_home = []
        self.arranged_data_away = []
        keys = ['GF15', 'GF5', 'SOT15', 'SOT5', 'Shot15', 'Shot5',
'Corner15', 'Corner5', 'FA15', 'FA5', 'GA5', 'Cleansheet', 'GA15',
'SOTA5', 'SOTA15', 'SA5', 'SA15', 'FF5', 'FF15', 'Card15', 'Card5',
'CPF15', 'CPF5']

        for team in self.teams:
            for source, side, target in [
                (self.teams_away_statistics_pure, 'away',
self.arranged_data_away),
                (self.teams_home_statistics_pure, 'home',
self.arranged_data_home)
            ]:
                for item in source:
                    if item['Team'] == team:
                        match_count = len(item['AS']) if side == 'away'
else len(item['HS'])
                        series = {key: [] for key in keys}
                        feature_matrix = []
                        for j in range(match_count):
                            row = self._feature_row_from_side_stats(item,
side, j)
                            feature_matrix.append(row)
                            for idx, key in enumerate(keys):
                                series[key].append(float(row[idx]))
                        target.append({
                            'Team': team,
                            'feature_matrix': np.array(feature_matrix,
dtype=float),
                            'next_feature_vector':
self._feature_row_from_side_stats(item, side, match_count),
                            'GF15': series['GF15'], 'GF5': series['GF5'],
                            'SOT15': series['SOT15'], 'SOT5':
series['SOT5'],
                            'Shot15': series['Shot15'], 'Shot5':
series['Shot5'],
                            'Corner15': series['Corner15'], 'Corner5':
series['Corner5'],
                            'FA15': series['FA15'], 'FA5': series['FA5'],
                            'GA15': series['GA15'], 'GA5': series['GA5'],
```

```
                            'Cleansheet': series['Cleansheet'],
                            'SOTA15': series['SOTA15'], 'SOTA5':
series['SOTA5'],
                            'SA15': series['SA15'], 'SA5': series['SA5'],
                            'FF15': series['FF15'], 'FF5': series['FF5'],
                            'Card15': series['Card15'], 'Card5':
series['Card5'],
                            'CPF15': series['CPF15'], 'CPF5':
series['CPF5']
                        })
                        break

        self.arranged_data_home = np.array(self.arranged_data_home,
dtype=object)
        self.arranged_data_away = np.array(self.arranged_data_away,
dtype=object)

    def build_final_x_matrix(self):
        rows = []
        X_list = []
        y_list = []

        for sample_id, (_, row) in
enumerate(self.all_matches_df.iterrows()):
            home_team = row['HomeTeam']
            away_team = row['AwayTeam']
            home_idx = int(row['season_idx']) * 19 +
int(row['home_side_season_idx'])
            away_idx = int(row['season_idx']) * 19 +
int(row['away_side_season_idx'])

            home_entry = self._find_entry(self.arranged_data_home,
home_team)
            away_entry = self._find_entry(self.arranged_data_away,
away_team)

            a_home_features = home_entry['feature_matrix']
[home_idx].reshape(1, -1)
            b_away_features = away_entry['feature_matrix']
[away_idx].reshape(1, -1)
            # Add bias term x_0 = 1 as the first column
            final_x_vector = np.concatenate([np.ones((1, 1)),
a_home_features, b_away_features], axis=1)
            y_int = self._label_to_int(row['FTR'], row['FTHG'],
row['FTAG'])
```

```
            item = {
                'sample_id': sample_id,
                'season_idx': int(row['season_idx']),
                'home_side_season_idx': int(row['home_side_season_idx']),
                'away_side_season_idx': int(row['away_side_season_idx']),
                'Date': row['Date'], 'HomeTeam': home_team, 'AwayTeam':
away_team,
                'FTHG': int(row['FTHG']), 'FTAG': int(row['FTAG']), 'FTR':
row['FTR'],
                'y_int': y_int, 'a_home_features': a_home_features,
                'b_away_features': b_away_features, 'x_vector':
final_x_vector
            }
            for idx, name in enumerate(self.home_feature_names):
                item[name] = float(a_home_features[0, idx])
            for idx, name in enumerate(self.away_feature_names):
                item[name] = float(b_away_features[0, idx])

            rows.append(item)
            X_list.append(final_x_vector[0])
            y_list.append(y_int)

        self.dataset_df = pd.DataFrame(rows)
        self.X = np.array(X_list, dtype=float)

        mask = np.ones(len(self.X), dtype=bool)

        for i in range(len(self.X)):
            if self.X[i, 5] == 0 or self.X[i, 28] == 0:
                mask[i] = False





        self.y_raw = self.dataset_df['FTR'].to_numpy()
        self.y_int = np.array(y_list, dtype=int)
        self.Y_one_hot = self._one_hot(self.y_int, num_classes=3)

        self.X = self.X[mask]
        self.y = self.y_int[mask]
        self.beta = np.zeros(47, dtype=float)
```

```
        self.X_splits = np.array_split(self.X, self.cross_validation)
        self.y_splits = np.array_split(self.y, self.cross_validation)


    def save_to_csv(self):

        self.X_no_bias = []
        self.mean = []
        self.std = []
        self.X_norm = []


        for i in range(self.cross_validation):
            self.X_no_bias.append(self.X_splits[i][:, 1:])  # 46 feature
            mean_i = self.X_no_bias[i].mean(axis=0)
            std_i = self.X_no_bias[i].std(axis=0, ddof=0)
            std_i[std_i == 0] = 1.0

            self.mean.append(mean_i)
            self.std.append(std_i)

            X_norm_i = np.concatenate([
                np.ones((self.X_splits[i].shape[0], 1)),
                (self.X_no_bias[i] - mean_i) / std_i
            ], axis=1)

            self.X_norm.append(X_norm_i)


            self.Y_zero_vs_rest.append(np.where(self.y_splits[i] == 0, 1,
0))
            self.Y_one_vs_rest.append(np.where(self.y_splits[i] == 1, 1,
0))
            self.Y_two_vs_rest.append(np.where(self.y_splits[i] == 2, 1,
0))

            x_columns = ['x_0'] + self.home_feature_names +
self.away_feature_names
            self.X_df = pd.DataFrame(self.X, columns=x_columns)
            self.X_norm_df = pd.DataFrame(X_norm_i, columns=x_columns)
            self.X_seperate_df = pd.DataFrame(self.X_splits[i],
columns=x_columns)
            self.y_df = pd.DataFrame({'y': self.y})
            self.y_seperate_df = pd.DataFrame({'y': self.y_splits[i]})
```

```
            self.X_df.to_csv(f'X_matrix.csv', index=False)
            self.y_df.to_csv(f'Y_matrix.csv', index=False)
            self.X_seperate_df.to_csv(f'X_matrix_fold{i}.csv',
index=False)
            self.X_norm_df.to_csv(f'X_norm_fold{i}.csv', index=False)
            self.y_seperate_df.to_csv(f'y_fold{i}.csv', index=False)









    def build_prediction_vector(self, home_team, away_team,
season_idx=None):
        home_team = self._resolve_team(home_team)
        away_team = self._resolve_team(away_team)
        if season_idx is None:
            season_idx = len(self.data_array) - 1

        home_stats = self._find_entry(self.teams_home_statistics_pure,
home_team)
        away_stats = self._find_entry(self.teams_away_statistics_pure,
away_team)

        home_matches_this_season = int(((self.all_matches_df['season_idx']
== season_idx) & (self.all_matches_df['HomeTeam'] == home_team)).sum())
        away_matches_this_season = int(((self.all_matches_df['season_idx']
== season_idx) & (self.all_matches_df['AwayTeam'] == away_team)).sum())

        home_idx = season_idx * 19 + home_matches_this_season
        away_idx = season_idx * 19 + away_matches_this_season

        a_home_features = self._feature_row_from_side_stats(home_stats,
'home', home_idx).reshape(1, -1)
        b_away_features = self._feature_row_from_side_stats(away_stats,
'away', away_idx).reshape(1, -1)
        final_x_vector = np.concatenate([a_home_features,
b_away_features], axis=1)
        return a_home_features, b_away_features, final_x_vector



    def run_all(self):
```

```
        self.load_data()
        self.calculate_scores_and_rates()
        self.process_match_statistics()
        self.handle_missing_seasons()

        self.arrange_data()
        self.build_final_x_matrix()
        self.save_to_csv()
```